Studio Nx

Backend(FastAPI/Python) 86 — AdminAuth.login()

2026-10-12

Backend(FastAPI/Python) 86 — `AdminAuth.login()` (관리자 화면 로그인)

항목	내용
파일	`studionx/backend/app/admin.py`, `requirements.txt`
작업	P1-21 공지 API와 관리자 화면: `/admin` 로그인 폼을 처리하는 함수 (PRD 01 FR-115)
작성	Claude (AI), 2026-10-13, 사용자 검토
이전 · 다음	85 공지 라우터 · 87 `AdminAuth.logout()`, `authenticate()` (다음)

1. 하는 일

`/admin`에서 이메일과 비밀번호를 보내면, **ADMIN 계정인 경우에만** 로그인으로 인정하고 세션에 관리자 번호를 저장합니다.

입력	결과
ADMIN 계정, 비밀번호 맞음	성공 (`True`), 세션에 `{"admin_user_id": 번호}`만 저장
일반 회원(`USER`), 비밀번호 맞음	실패 (`False`), 세션은 비어 있음
비밀번호 틀림, 없는 이메일, 빈 폼	실패
정지(`SUSPENDED`) 또는 탈퇴(`DELETED`)한 관리자	실패
이메일에 공백이나 대문자가 섞임	일반 로그인처럼 정리해서 확인 (성공 가능)

실패 이유는 화면에 알려 주지 않습니다. 모두 같은 "잘못됨" 한 가지로 보입니다.

2. 언제 쓰나

호출하는 곳	화면
SQLAdmin이 `/admin/login` 폼 전송을 받을 때	관리자 로그인 화면

3. 코드

import (이번에 추가한 것)

```
from app.core.errors import AppError
from app.db.session import SessionLocal
from app.models.user import UserRole
from app.services import auth_service
```

함수

```
async def login(self, request: Request) -> bool:
    form = await request.form()
    email = str(form.get("username") or "")
    password = str(form.get("password") or "")
```

```
    db = SessionLocal()
    try:
        user = auth_service.authenticate(db, email, password)
        if user.role != UserRole.ADMIN:
            return False
        request.session.update({"admin_user_id": user.user_id})
        return True
    except AppError:
        return False
    finally:
        db.close()
```

백엔드(Python)는 지금 방식 그대로여서 "최신 문법" 버전은 따로 두지 않습니다.

패키지: `requirements.txt`에 `itsdangerous>=2.1`을 추가했습니다. 세션 쿠키를 서명하는 데 쓰는 패키지이며, 없으면 `AdminAuth`를 만드는 순간 `ModuleNotFoundError`가 납니다. **다른 컴퓨터에서는 pull 뒤에 `pip install -r requirements-dev.txt`를 다시 실행해야 합니다.**

문법 풀이

식	뜻
`await request.form()`	로그인 폼에서 보낸 값을 읽습니다. SQLAdmin의 로그인 폼은 입력 칸 이름이 `username`, `password`입니다
`str(form.get("username") or "")`	칸이 없거나 비어 있어도 빈 글자로 바꿔서, 파일이나 `None`이 들어와도 오류가 나지 않게 합니다
`SessionLocal()`	이 함수가 직접 DB 세션을 엽니다. SQLAdmin은 FastAPI의 `Depends`를 쓰지 않아서 `Db`를 받을 수 없습니다
`auth_service.authenticate(db, email, password)`	일반 로그인(P1-05)과 같은 확인입니다. 이메일 정리, 비밀번호 확인, 탈퇴와 정지 계정 거절, 마지막 로그인 시각 기록까지 합니다
`except AppError`	확인에 실패하면 서비스가 `AppError`를 던지므로, 모두 `False`로 바꿉니다
`finally: db.close()`	성공하든 실패하든 DB 세션을 닫습니다. `return`이 `try` 안에 있어도 `finally`는 실행됩니다
`request.session.update({...})`	SQLAdmin이 서명해서 쿠키로 보관하는 세션에 값을 저장합니다

4. 흐름

1. 폼에서 이메일과 비밀번호를 읽습니다.
2. 일반 로그인과 같은 `authenticate`로 확인합니다. 실패하면 `AppError`이므로 `False`입니다.
3. 확인된 회원의 `role`이 `ADMIN`이 아니면 `False`입니다.
4. 맞으면 세션에 `admin_user_id`를 저장하고 `True`입니다.
5. 어느 경우든 DB 세션을 닫습니다.

5. 왜 이렇게

선택	이유
새로 확인 코드를 만들지 않고 `authenticate`를 재사용	비밀번호 확인, 없는 이메일의 응답 시간 맞추기, 탈퇴와 정지 거절을 한 곳에서 관리합니다. 관리자 로그인에서만 다르게 동작할 이유가 없습니다

선택	이유
일반 회원은 비밀번호가 맞아도 `False`	비밀번호가 맞다는 것과 관리자라는 것은 다릅니다. 권한은 `role`로 따로 확인합니다
실패 이유를 구분해서 알리지 않음	"이 이메일은 관리자가 아님" 같은 정보로 관리자 계정을 찾지 못하게 합니다
세션에 번호 하나만 저장	비밀번호나 역할을 쿠키에 넣지 않습니다. 매 요청마다 DB에서 다시 확인합니다 (87번 `authenticate`)
API의 JWT와 세션 쿠키를 따로 씀	관리자 화면은 SQLAdmin이 자체 세션으로 관리하고, API의 토큰과 섞이지 않습니다
관리자 계정은 DB에서 직접 `role = 'ADMIN'`으로 지정	화면에서 관리자를 만드는 기능이 없습니다 (뼈대 설명 그대로)

6. 시험 (`tests/test_admin_auth.py`, 7개)

시험	결과
ADMIN 계정: `True`, 세션이 `{"admin_user_id": 번호}` 하나뿐	통과입니다
이메일에 공백과 대문자가 섞여도 성공	통과입니다
일반 회원은 비밀번호가 맞아도 `False`, 세션은 비어 있음	통과입니다
틀린 비밀번호, 없는 이메일, 빈 폼	모두 `False`
정지된 관리자, 탈퇴한 관리자	모두 `False`, 세션은 비어 있음
`login`이 DB 세션을 직접 열고 한 번 닫음	통과입니다

7개 모두 통과했습니다. `ruff`도 통과했습니다.